Концепт на вештачка интелигенција преку играта „икс-нула“

Бојан Дамјановски, Ментор: Мартин Динев
Март, 2025

Апстракт

Овој труд ја истражува примената на невроеволуција за тренирање на агенти во игри за двајца со нулта сума, при такви услови оптималната стратегија да не е достапна па не може да се искористи учењето со надзор. Карактеристично е што агентите се тренираат на самите себеси играјќи ја играта, во овој случај „икс-нула“. Исто така, испробуваме нов метод на подредување на агентите инспириран од натпревари по шах кој има мала случајност во него. Го разгледавме влијанието на разните варијабли на невроеволуцијата и демонстриравме успешно развиен агент кој ги разбира условите за победа и загуба.

I. Вовед

Меѓу карактеристичните типови на игри во теоријата на игрите се игрите за двајца со нулта сума, односно игри во кои мора да има губитник за да има победник. Во случај да ни е потребно да тренираме агент кој наоѓа поволна стратегија за ваква игра, ако претпоставиме дека оптималната стратегија не ни е достапна, па немаме како да го употребиме ефективниот метод на учење со надзор, кому во прилог со влезните информации му се потребни точните одговори за кои агентот треба да се прилагоди. [5] Една од алтернативите која се користи во овие случаи е невроеволуцијата.[6] Невроеволуција е генетски алгоритам, кој наликува на еволуција, така што хромозомот на еден агент, чија поволност се проверува, претставува невронска мрежа.

Целта е да експериментираме со варијаблите на невроеволуцијата на таква игра, во овој случај „икс-нула“. Во „Икс-нула“ двајца играчи, еден по друг, се натпреварува да постават жетони, X и 0 соодветно, трипати во права линија на квадратно поле со димензии 3x3. Иако играта е решена[7], поради кратка должина, малото играчко поле и сличноста на улогите на двата играча, ја одбравме „икс-нула“ за истражувањето. Клучно за истражувањето е што агентите се тренираат меѓу себе и што моделот кој го градиме не зависи од икс-нула и е употреблив во други проекти.

II. Планот за експериментите

Прво ќе го имплементираме потребниот софтвер: Модел на агент-играч, Табла на која ќе се игра, како и начин на рангирање преку т.н. рунди и турнири. Кодот можете да го најдете во [1]. Потоа ќе спроведеме компаративна анализа во однос на: Избор на невронска мрежа; Колкав дел од популацијата да продолжи во следната генерација, и Структурата на турнирот, односно начинот на рангирање. На крај ќе ја употребиме најдобро покажаната комбинација и ќе ги анализираме резултатите.

III. Моделот на агент-играч

Моделот на агент е моделиран според концептот на Невроеволуција. Се состои од долга низа, хромозом, од броеви, гени, кој ги репрезентираат коефициентите во невронска мрежа. Невронската мрежа се состои од 9 влезни податоци, секое за по едно поле од таблата така што 1 означува свој жетон, -1 туѓ и 0 означува празно поле. Излезот е исто така 9 податоци, колку е сигурен во проценти дека на тоа поле треба да стави жетон. На највисоко рангираното поле поставува жетон. Во интерес на времето, ако се обиде да постави жетон на зафатено поле добива пенал во поените и се зема второто рангирано поле итн. Дополнително има два скриени слоја со димензии 8 и 4 соодветно, и за активациска функција ја зедовме $f(x) = \frac{1}{1+e^{-x}}$.

IV. Имплементација на Таблата, Рунди и Бодување

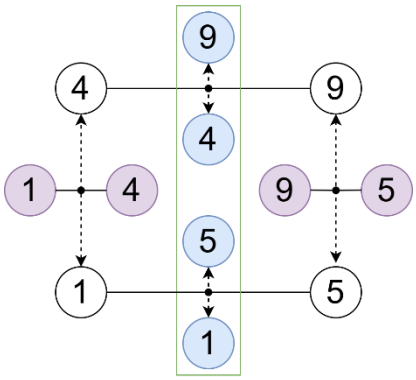
За да се споредат агентите треба да обезбедиме натпревари меѓу нив. Овие ќе ги нарекуваме рунди. Една рунда се состои од две игри на икс-нула, така и двата агента почнуваат први по еднаш. Рундата е одговорна за комуникација меѓу агентите и таблата: агентите треба да добијат влезни податоци, а излезните треба да се толкуваат и соодветното поле да биде зафатено. Рундата е исто така одговорна за доделување поени на агентите кога ќе победат, како што и пенали кога ќе направат грешка. Победникот добива 2 поени, губитникот -2, и секоја грешка, т.е. избор на зафатено поле, се казнува со -1 поен.

V. Турнири т.е. Рангирање

Генетскиот алгоритам работи така што се мери поволност на секој член на генерацијата, потоа се рангираат според поволноста, и најдобрите се репродуцираат, т.е. гените им се мешаат и произведуваат нови агенти. Старите најдобри, нивните деца, како и неколку нови случајно генерирани агенти се членовите на новата генерација, која повторно се рангира итн. сè додека не сме задоволни. [6]

Процесот на рангирање го правиме преку турнири. Турнирите ги одржуваат рундите меѓу агентите од генерацијата, ги следат нивните резултати, и ги подредуваат според нивната поволност. Агентот А се рангира повисоко од Б ако го победи во рунда.

Ќе споредиме два алгоритми за подредување. Првиот е стандардниот алгоритам за подредување во Python, т.н. Tim Sort. (документација: [4]) Вториот го базиравме на турнир по шах. Агентите се поделени по парови и играат рунда, потоа победниците се подредени по парови и тие играат итн., истото се прави и со губитниците. На крај агентот кој секогаш победил е прв, оној кој последно играл со него е втор, па двата што играле со нив се трет и четврт во зависност од нивната последна рунда, која ја играле еден против друг, и така сè до последниот. На *Слика 1* е илустративно прикажан пример (броевите 1,4,9,5, се подредени во 9,4,5,1). Важно е да се забележи дека ова подредување не е перфектно и се надеваме дека на тоа ќе има позитивен ефект врз алгоритамот.



Слика 1. Пр. модел на турнир.

Начинот на определување на поволноста на еден агент е клучна за успехот на еден генетски алгоритам.[6] Во нашиот случај, бидејќи целиме да направиме модел кој би работел и за други игри, не само икс-нула, сме ограничени поволноста да не зависи од играта. Решивме поволноста на агент да е процентот од можните поени кој го освоил.

VI. Експеримент 1: Избор на невронска мрежа

Направивме споредба помеѓу едноставна рачно напишана невронска мрежа користејќи го методот „ширење нанапред“ и познатата невронска мрежа од библиотеката „scikit learn“, т.н. „MLPRegressor“. (документација: [3]) Го споредивме просечното време потребно да се генерираат со случајни вредности и да пресметаат излезни податоци од вкупно 500 итерации. Резултатите се прикажани на *Табела 1*.

Модел	Просечно потребно време
MLPRegressor	0.04820874s

Нашиот модел	0.00064126s

Табела 1. Споредба на невронски мрежи

Нашиот модел, кој нема никакви оптимизации, е речиси 100 пати побрз, веројатно бидејќи MLPRegressor-от е наменет за тренирање преку учење со надзор, заради кое е потребно повеќе време за генерирање.[3]

VII. Експеримент 2: Процентот од популацијата што преживува

Направивме споредба меѓу три случаи, да „преживее“ 40%, 60% и 80% од популацијата. Оние кои преживуваат потоа се репродуцираат и со неколку нови, случајно генерирани агенти ја претставуваат новата генерација. Резултатите се на *Графикон 1*:

- Големината на популација е 2^7
- Извршени се 1000 генерации.
- Рангирани се преку подредување.

X-оската е генерацијата, Y-оската е просекот на поволност на најдобрите 25% од агентите, т.е. просек на првиот квадрант (озн.: Q1M). Го споредуваме тоа бидејќи во пракса би се користеле најдобрите агенти од која било генерација. Истото ќе важи за идните графикони 2 и 3.

Графикон 1. Q1M според дел од популацијата која преживува

Од резултатите е тешко да се заклучи која од опциите е најдобра. Опцијата со 80% покажува малку, но не значително, подобри резултати од останатите. Сепак ќе ја користиме во идните експерименти.

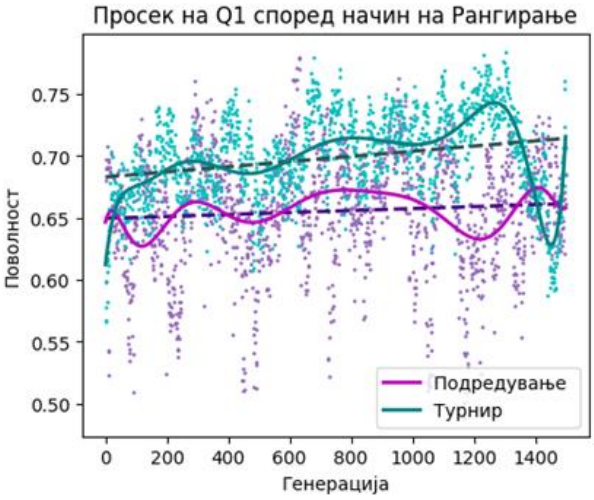
VIII. Експеримент 3: Начин на рангирање

На *Графикон 2* се резултатите според начините на рангирање:

- Големината на популација е 2^7
- Извршени се 1500 генерации.
- Преживува 80% од популацијата.

Го зголемивме бројот на генерации за подобро да се гледа разликата.

Резултатите се јасни, моделот на шаховски турнир се покажа подобар, и ќе го користиме понатаму. Претпоставуваме дека оваа непрецизност дозволува да има поразновидни генерации, што влијае добро врз генетскиот алгоритам.[2]



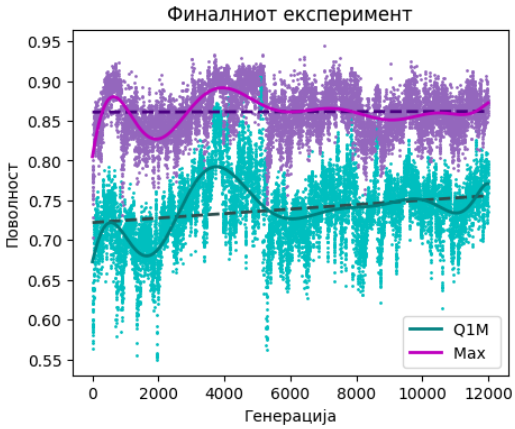
Графикон 2. Q1M според на начин на рангирање

IX. Финален Експеримент и Дискусија

Земајќи ги предвид претходните експерименти, одново го извршивме алгоритамот до 12000. генерација. Резултатите се на *Графикон 3*:

- Големината на популација е 2^7
- Извршени се 12000 генерации.
- Преживува 80% од популацијата.
- Рангирани се преку турнир.

Претставени се просекот на поволност најдобрите 25% од агентите како и поволноста на најдобриот агент од секоја генерација.



Графикон 3. Финалниот експеримент

На популациско ниво се забележува генерално подобрување во поволност обележано од растот на Q1M, кој со текот на генерациите се доближува до максимумот. Со ова, без да навлегуваме во квалитетот, сметаме дека моделот е успешен.

За да ги провериме резултатите игравме неколку рунди со највисоко рангираниот агент од последната генерација. Во повеќето комбинации приоритет на агентот му беше приоритет да спречи пораз, дури и на сметка на сопствена победа. Во секој случај, се целеше кон редење на 3 жетони по ред, од што може да се заклучи дека ја знае целта.

Слаба точка на нашиот модел, кој е дизајниран за лесна повторна употреба, е тоа што поволноста е релативна меѓу генерациите и не може да се промени тоа без да се разгледува нешто специфично за самата игра, со што би се изгубила општоста на моделот. Сепак, самиот факт што поволноста варира укажува дека алгоритамот е жив, т.е. нема предвреме конвергирано, што е задоволително според нашите стандарди.

X. Заклучок

Најклучно може да се заклучи дека користење на MLPRegressor е грешна одлука. Освен тоа, модел на невроеволуција каде агенти се тренираат меѓусебно, користејќи го нашиот модел на турнир, се покажа успешно. Развивме агент кој делумно ги препознава критериумите за победа и загуба. Со овој труд демонстриравме дека невроеволуцијата е задоволителна алтернатива кога не е достапна стратегија за тренирање на агенти.

Литература:

[1] Дамјановски, Б. Kod za Proektnata Matura. GitHub: https://github.com/bojandam/ProektnaMatura

[2] A. L. Nsakanda, W. L. (2007). Ensuring population diversity in genetic algorithms: A technical note with application to the cell formation problem European Journal of Operational Research, Volume 178, Issue 2, 634-638.

[3] MLPRegressor. Retrieved from Scikit learn: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html

[4] Python Software Foundation. Python 3.13.2 documentation. https://docs.python.org/3/

[5] M. Mohri, A. Rostamizadeh, A. Talwalkar (2012) Foundations of Machine Learning, The MIT Press ISBN 9780262018258.

[6] Stanley, Kenneth O. (2017-07-13). "Neuroevolution: A different kind of deep learning". O'Reilly Media. Retrieved 2017-09-04.